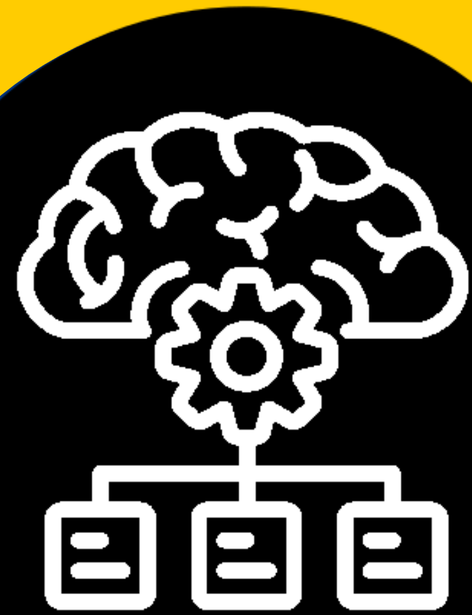
Deep Learning

Convolutional Neural Networks

Research Depth and Industry Skills

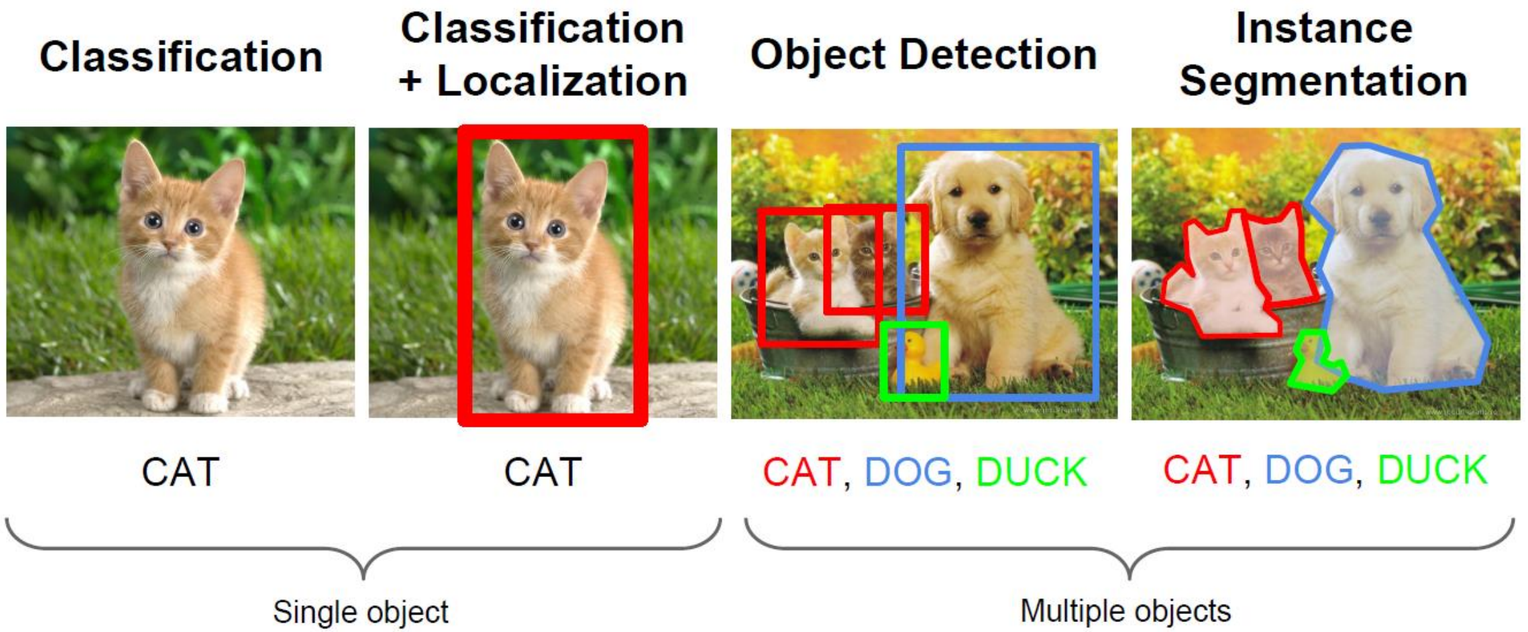


By:

Dr. Zohair Ahmed

Computer Vision Tasks

- Computer vision has been the primary area of interest for ML
- The tasks include: classification, localization, object detection, instance segmentation



What computers ‘see’: Images as Numbers

1. An image is just numbers

- A computer doesn’t “see” a picture the way humans do.
  It only sees **a grid (matrix) of numbers**.
- For a **1080 × 1080** RGB image:
  - Height = 1080 pixels
  - Width = 1080 pixels
  - Channels = 3 (Red, Green, Blue)
- So the image is basically:
- 1080 x 1080 x 3 matrix
- The 3 is the number of channels. Each value is between 0 to 255
- Each number tells how bright the color is at that pixel.

2. Can we just flatten the image into a long vector and classify?

- Flattening means turning the image into **one long list of numbers**, for example:
- 1080 × 1080 × 3 = 3,499,200 values in one vector!
  - [1 2
  - 3 4]
  - After flatten: [1, 2, 3, 4]
- Flatten simply reshapes a multi-dimensional matrix into one long 1-D vector.
- A neural network could try to classify this, **but it would fail** or be extremely inefficient because:

Why this is a bad idea?

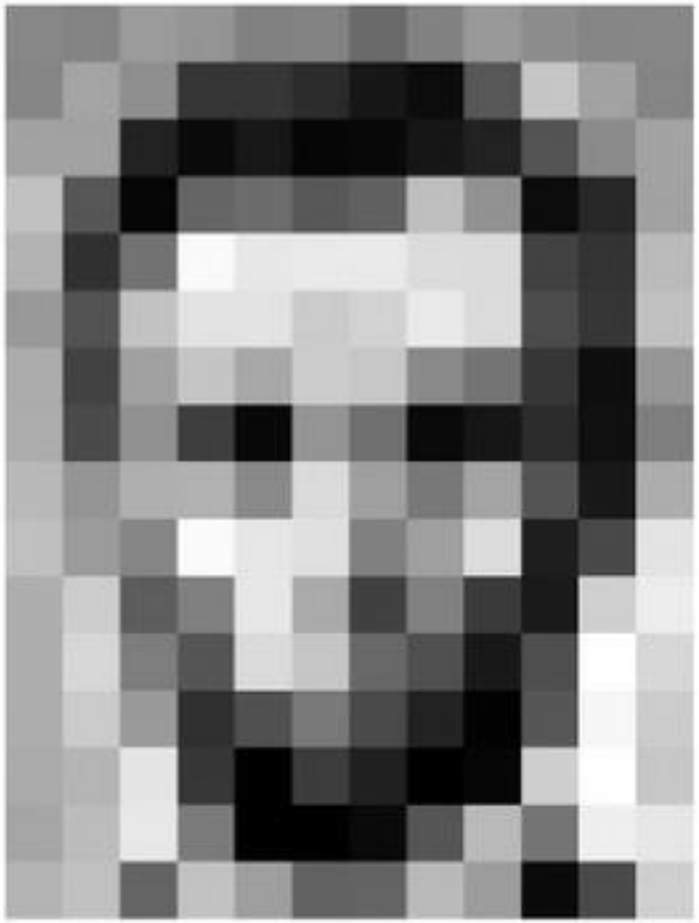
- It ignores **spatial structure** (the arrangement of pixels)
  - Imagine this small image:
  - [[0, 0, 0],
  - [0, 255, 0],
  - [0, 0, 0]]
  - Here the white pixel is in the **center**.
  - If you flatten it: [0, 0, 0, 0, 255, 0, 0, 0, 0]
  - When flattening, that 2D/3D layout is lost as structure, even though the values are still there
- Nearby pixels usually relate to each other (edges, shapes)
- A flat vector loses all this information
- Too many inputs → huge number of parameters → overfitting, slow training
- So a normal classifier (like a dense feed-forward NN) is **not appropriate**.

Eight Digit as Number

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0	
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29	
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0	
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1	
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49	
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36	
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62	
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0	
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0	
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19	
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0	
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0	
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4	
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0	
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0	
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3	
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0	
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4	
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5	
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0	
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1	
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0	

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0	
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29	
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0	
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1	
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49	
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36	
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62	
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0	
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0	
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19	
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0	
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0	
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4	
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0	
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0	
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3	
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0	
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4	
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5	
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0	
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1	
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0	

Images as Numbers



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

RGB Channel

1. A 3-channel image (RGB)

- If an image is 64 × 64 × 3, that means:
  - Height = 64 pixels
  - Width = 64 pixels
  - Channels = 3 (R, G, B)
- So each pixel is not one number.
- Each pixel is an array of 3 numbers:
  - [pixel_red_value, pixel_green_value, pixel_blue_value]
- Each value is between 0–255.

2. Grid structure

- A 64×64 RGB image is like a grid of 64 rows × 64 columns.
- Each cell in the grid = a pixel
- Each pixel contains a 3-element array.
- [[[R,G,B], [R,G,B], [R,G,B], ... 64 times] ← Row 1
- [[R,G,B], [R,G,B], [R,G,B], ... 64 times] ← Row 2
- ...
- 64 rows
-]

3. What is inside the top-left pixel?

- The top-left pixel is located at (row 0, col 0):
- Its value looks like:
  - image[0][0] = [R_value, G_value, B_value]

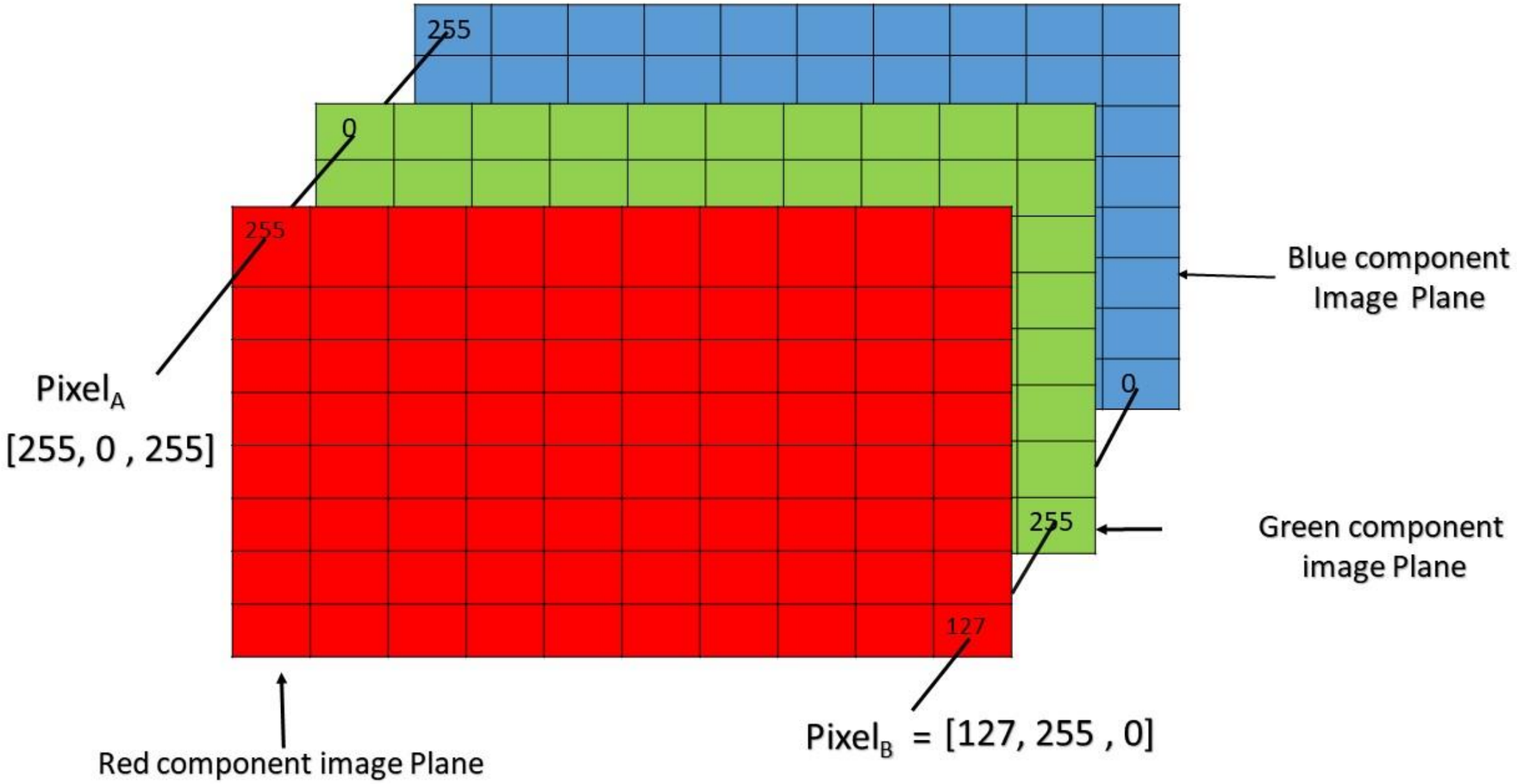
For example:

- image[0][0] = [123, 45, 200]
  - Red level = 123
  - Green level = 45
  - Blue level = 200

RGB Channel

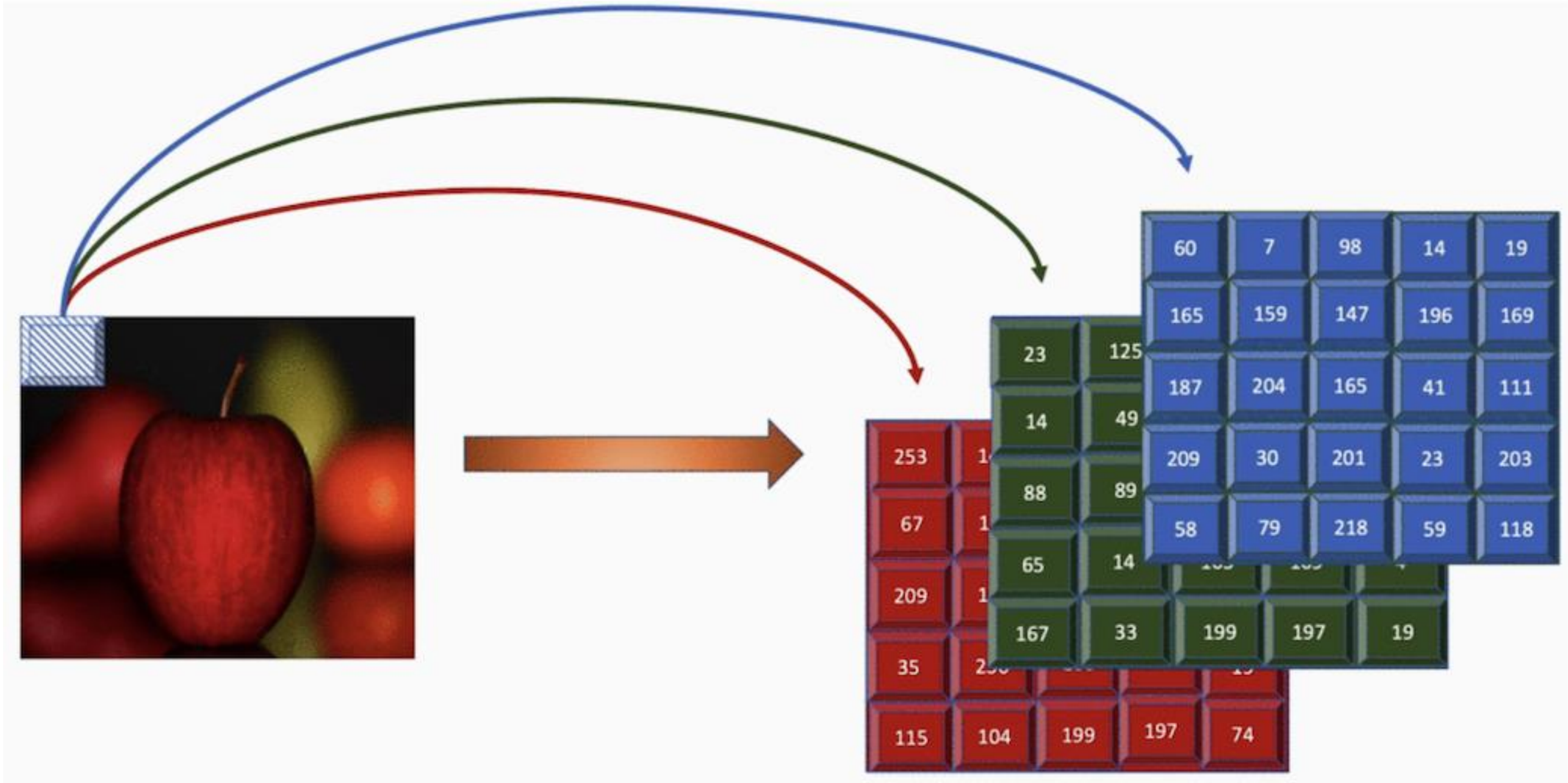
- Each row has 64 columns, and each cell is exactly one pixel in the image grid.
- Row = 64 pixels
- Column = 64 pixels
- Each cell = 1 pixel = [R, G, B] values
- A pixel has 3 values (Red, Green, Blue), each between 0–255.
- Here is one sample pixel: [120, 45, 200]
  - R = 120 (medium red)
  - G = 45 (low green)
  - B = 200 (high blue)
- This pixel will look purplish, because blue is high and green is low.

RGB Channel



Pixel of an RGB image are formed from the corresponding pixel of the three component images

RGB Channel


60 7 98 14 19
165 159 147 196 169
187 204 165 41 111
209 30 201 23 203
58 79 218 59 118
23 125
14 49
88 89
65 14
167 33 199 197 19
253
67
209
35
115 104 199 197 74

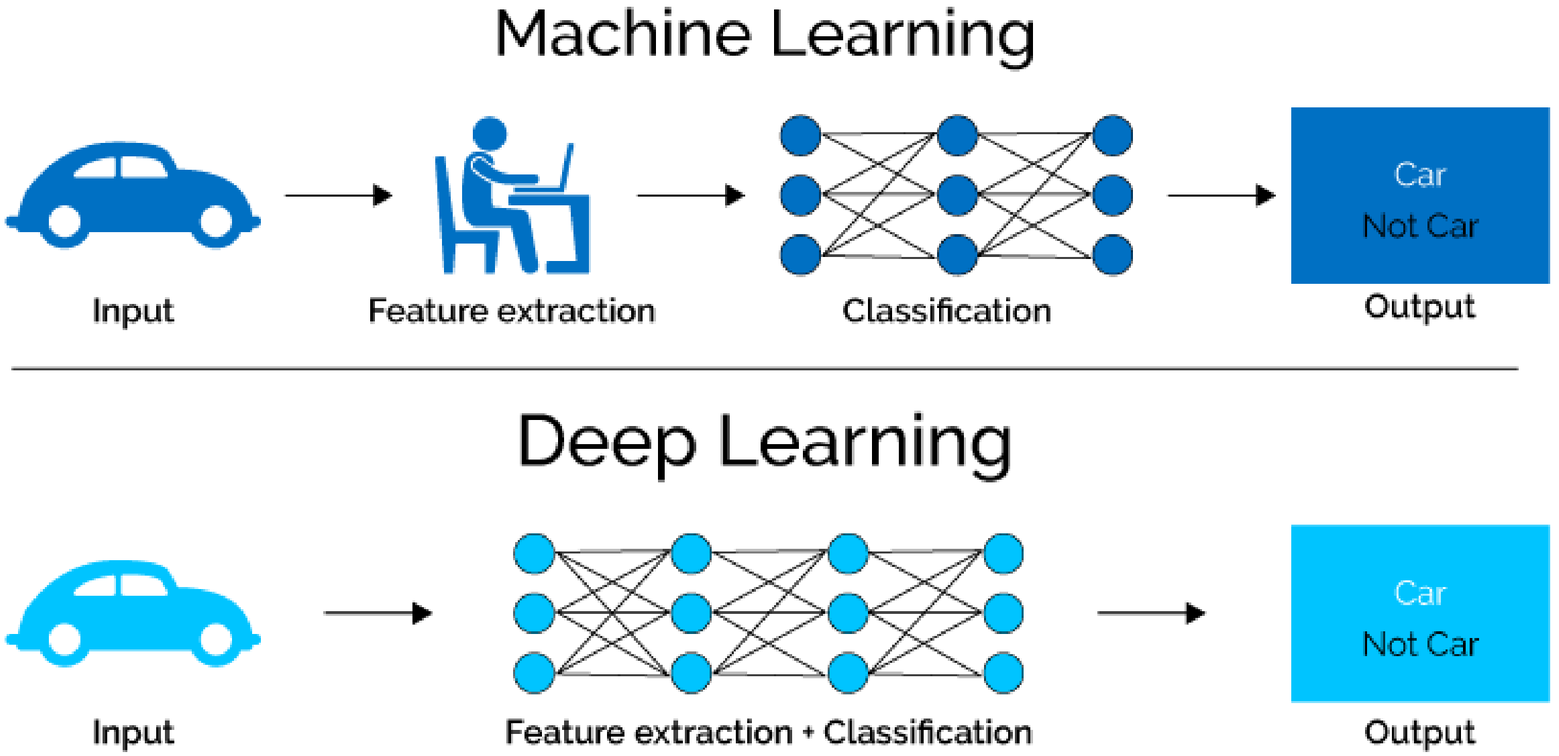
Need: Convolutional Neural Networks (CNNs)

CNNs take advantage of the fact that images have **local patterns**:

- **Learn small patches**
  Filters (like 3×3 or 5×5) scan the image and learn:
  - edges
  - corners
  - small textures
- **Combine small patterns into bigger patterns**
  Deeper layers learn:
  - eyes
  - nose
  - mouth
  - facial structure
  - identity-specific features
- **Finally classify the face**
  After extracting features, the network decides:
  - Lincoln?
  - Washington?
  - Jefferson?
  - Obama?
- **Why CNNs work?**
- They preserve **spatial relationships**
- They detect **local patterns**
- They reuse the **same filter** everywhere → fewer parameters
- They build hierarchical features from simple → complex

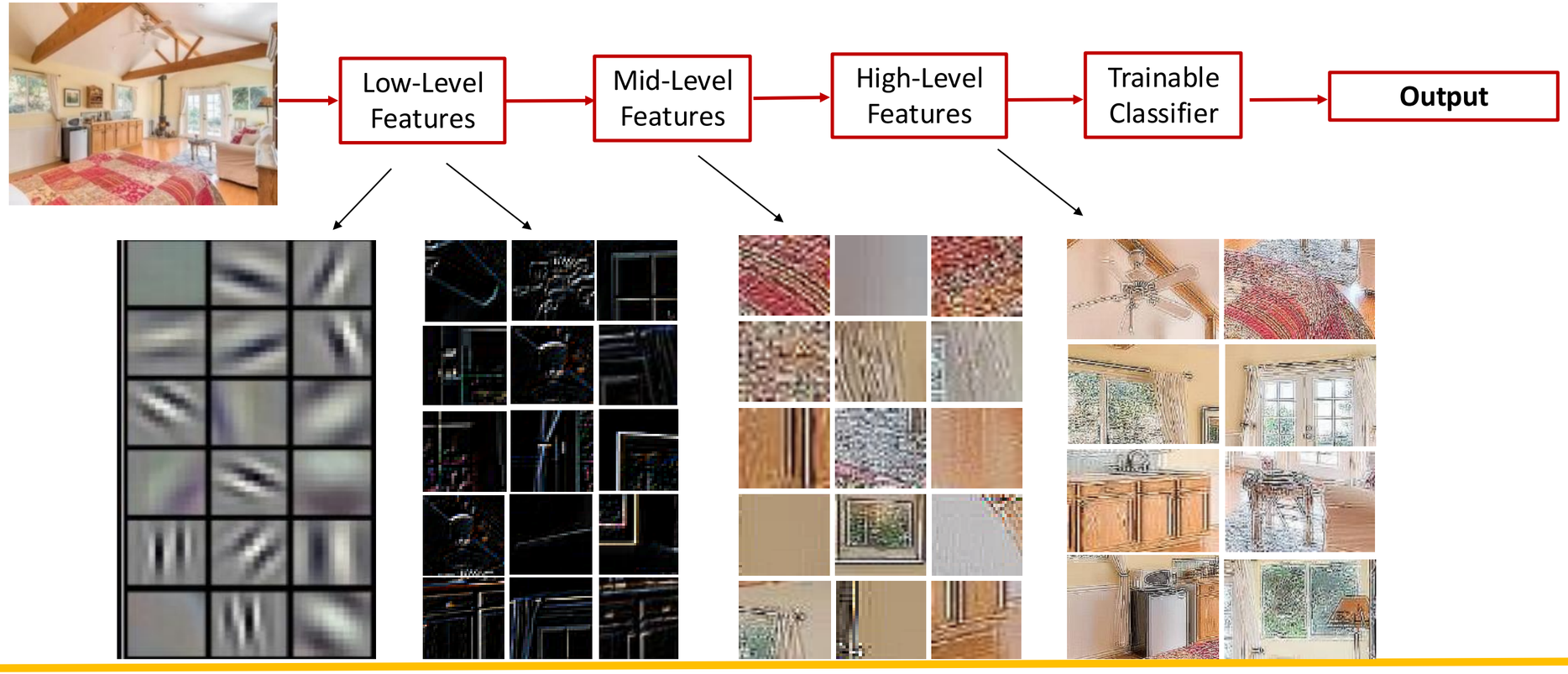
Deep Learning vs Machine Learning

- Deep learning (DL) is a machine learning subfield that uses multiple layers for learning data representations
- DL is exceptionally effective at learning patterns



Deep Learning

- DL applies a multi-layer process for learning rich hierarchical features (i.e., data representations)
  - Input image pixels → Edges → Textures → Parts → Objects



Convolutional Neural Networks (CNNs)

- A **CNN** is a type of **neural network** designed especially for **images**.
- A normal neural network treats input like a long list of numbers.
- A **CNN** is different because it looks at **small local parts of the image** first, such as:
  - edges
  - corners
  - textures
  - simple shapes
- That is why CNNs work very well for:
  - image classification
  - object detection
  - face recognition
  - medical image analysis

Neural Network with a Convolution Operation

- This means a CNN has at least one layer that uses **convolution** instead of only regular dense connections.

Convolution means:

- Take a **small filter/kernel**
- Slide it over the image
- At each position, multiply filter values with image values
- Add them up
- Produce a new value
- This helps the network detect patterns.

Neural Network with a Convolution Operation

- **Small example of convolution**
- Suppose part of an image is:
- [[1, 2, 0],
- [0, 1, 3],
- [2, 1, 0]]
- and a filter is:
- [[1, 0],
- [0, 1]]
- Apply the filter to the top-left 2×2 part:
- [[1, 2],
- [0, 1]]
- Now multiply element by element and add:
- (1×1) + (2×0) + (0×0) + (1×1) = 2
- That gives one output value.
- Then the filter slides to the next location and repeats.

Deep Feed Forward ANN

Feed-forward means:

- Data moves in one direction:
  - Input → Hidden Layers → Output
- **Deep means:**
- The network has **many layers**, not just one or two.

Example CNN flow:

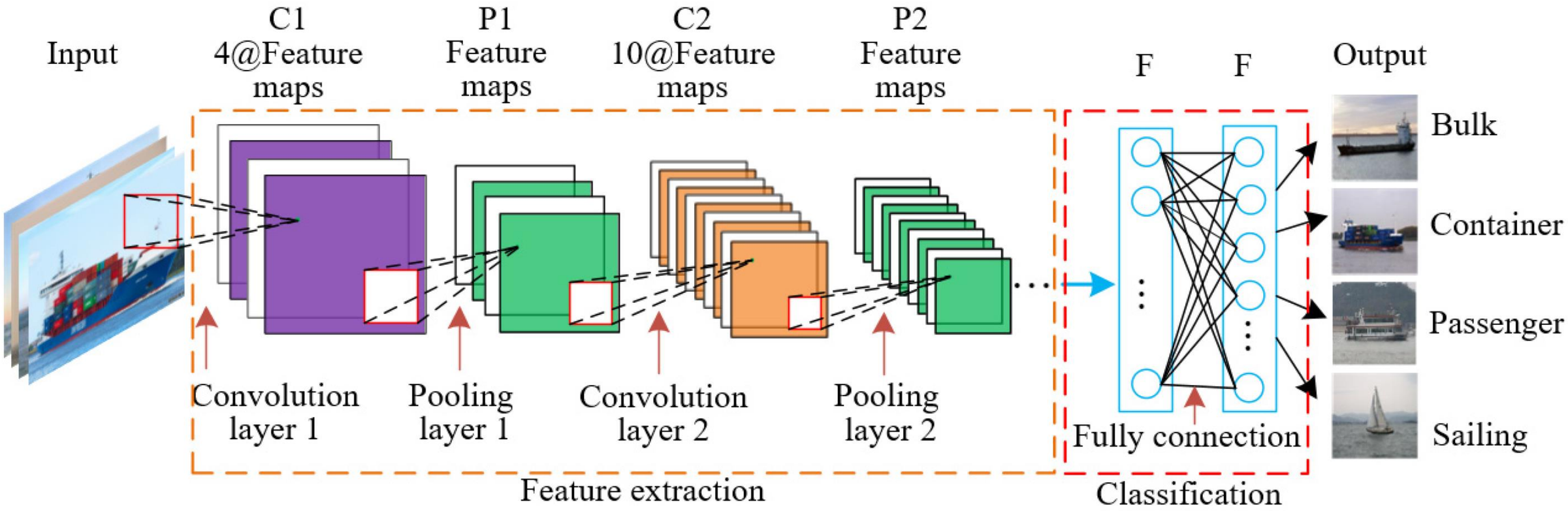
- Image → Convolution → ReLU → Pooling → Convolution → Flatten → Dense → Output

Example pipeline CNN

- Suppose input image shape is: (32, 32, 3)
- A CNN may do:
- **Convolution**
  - detect edges
- **ReLU**
  - keep useful activations
- **Pooling**
  - reduce size
- **Convolution**
  - detect more complex features
- **Flatten**
  - convert to vector
- **Dense**
  - classify image

CNN Architecture

A CNN is a deep feed-forward neural network that uses convolution layers to automatically learn spatial features from images.



Convolutional Neural Networks (CovNet)

- A ConvNet usually has 3 types of layers:
- 1) **Convolutional Layer** (*CONV*)
- 2) **Pooling Layer** (*POOL*)
- 3) **Fully Connected Layer** (*FC*)

CNN
Structure

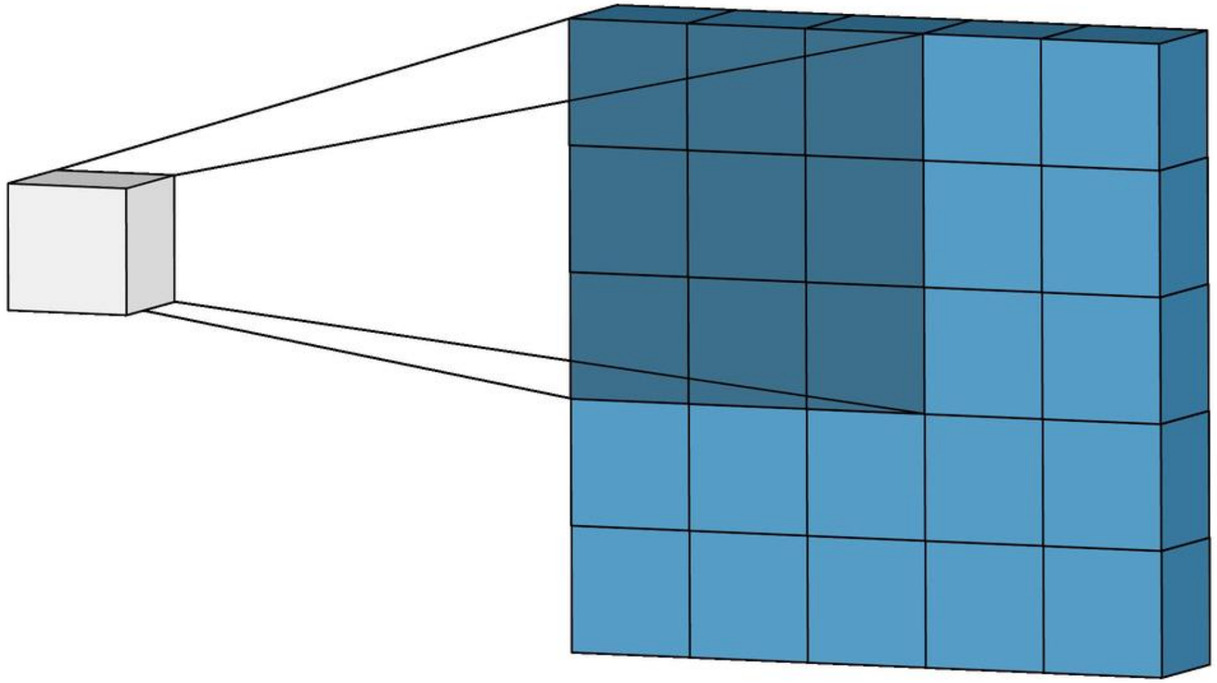
Convolutional Neural Networks

1. Filter / Kernel

A **filter** is a **small matrix of numbers** that slides over the image to find patterns.

- It can learn things like:
  - edges
  - corners
  - textures
  - simple shapes
- Suppose image part is:
  - [1 2 3]
  - [4 5 6]
  - [7 8 9]
- A filter may be:
  - [1 0]
  - [0 1]
- The filter looks at a small area of the image and does:
  - multiply
  - add
  - produce one output value
- So, a filter is like a **small pattern detector**.
  A filter is like a **tiny scanner** checking the image piece by piece.
- **A feature map (or activation map)** is the output generated when a convolutional filter (kernel) slides across an input image, highlighting detected patterns like edges, textures, or shapes.

1. Filter / Kernel



1. Filter / Kernel (Calculate Values)

- **Image patch (3×3)**
- **Filter (3×3)**
- Multiply element-wise
- Sum everything
- Get **one output value**
- This is called **convolution**.
- **Step-by-step Filter Calculation Example**
- 🖼 **Image patch (3×3)**
  - 1 2 3
  - 4 5 6
  - 7 8 9
- 🔍 **Filter (3×3)**
  - 1 0 1
  - 0 1 0
  - 1 0 1
- **Step 1: Multiply element-wise**
- We multiply each position:
  - (1×1) + (2×0) + (3×1)
  - (4×0) + (5×1) + (6×0)
  - (7×1) + (8×0) + (9×1)

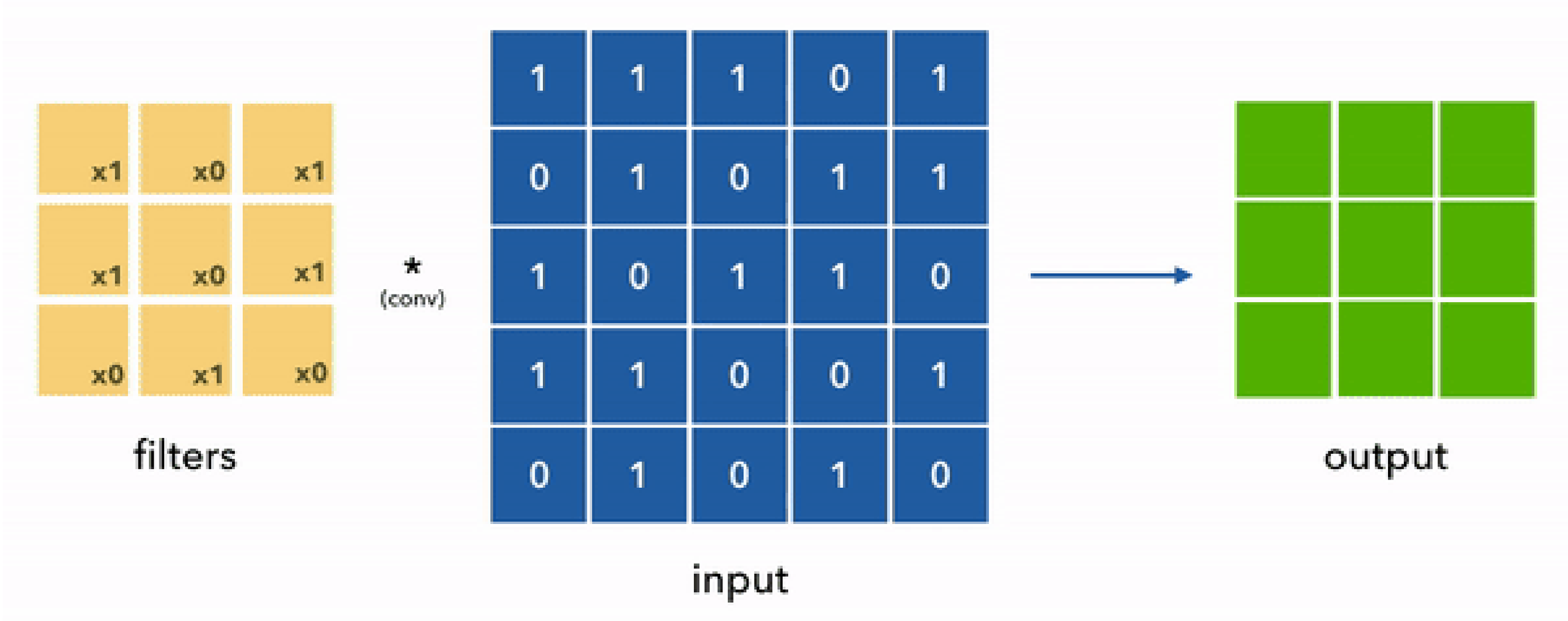
Step 2: Compute each multiplication

- 1 + 0 + 3
- 0 + 5 + 0
- 7 + 0 + 9

Step 3: Sum all numbers

- Add everything:
- 1 + 0 + 3 + 0 + 5 + 0 + 7 + 0 + 9
- Now sum: = 25
- **Final Output Value (1 cell in feature map) 25**
- This is exactly one value in the **feature map**.

1. Filter / Kernel (Calculate Values)



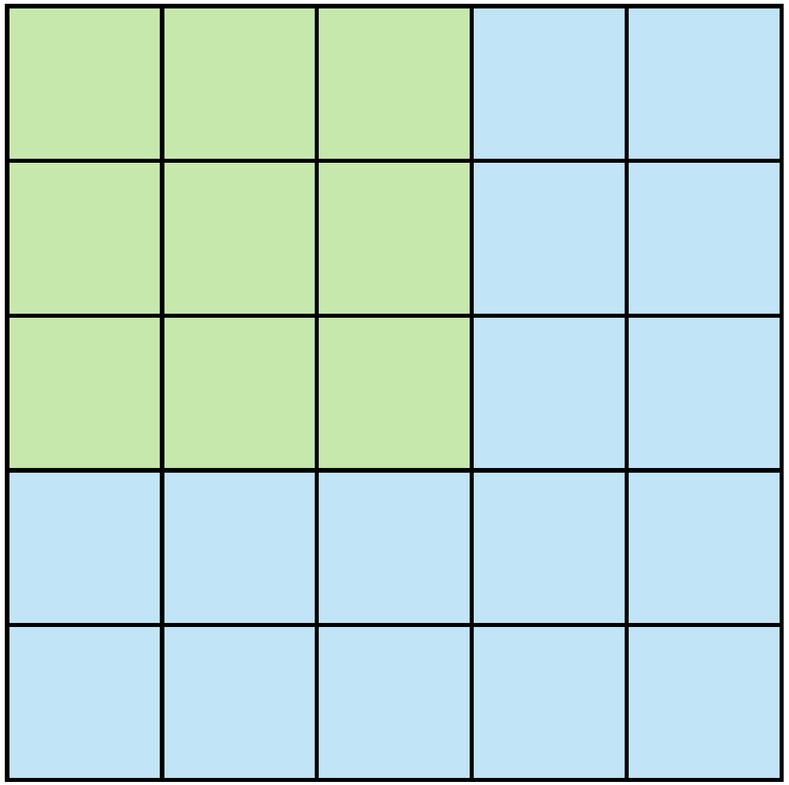
2. Stride

- **Stride** means **how many steps the filter moves each time**.
- Example
- If stride = **1**:
  - move 1 pixel at a time
- If stride = **2**:
  - move 2 pixels at a time
- **small stride** → more detailed scanning
- **large stride** → faster, but may miss details

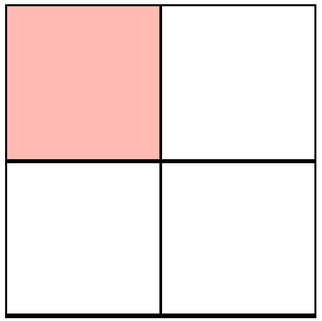
Example

- If filter starts here:
  - step 1 -> position 1
  - step 2 -> move right
  - step 3 -> move right again
- With:
  - **stride 1** = move little by little
  - **stride 2** = jump more

2. Stride (n=2)



Stride 2



Feature Map

2. Stride (How Different Stride make Conv Maps)

- **Image size:** 6 × 6
- **Filter size:** 3 × 3
- **Stride:** 1

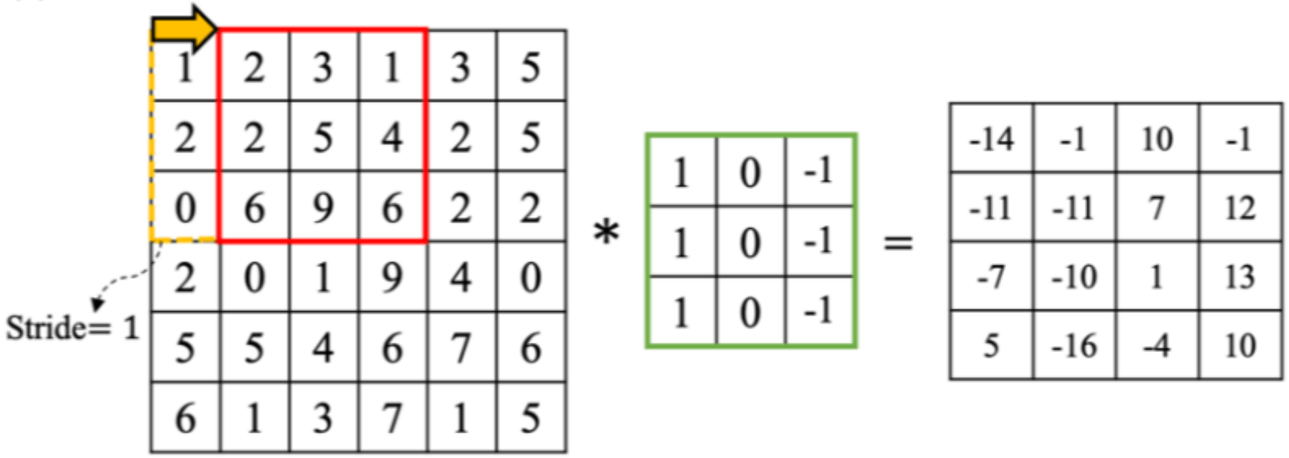
Compute Output Size:

- **Output size** $= \frac{n-f}{s} + 1$
- n=6 (image size)
- f=3 (filter size)

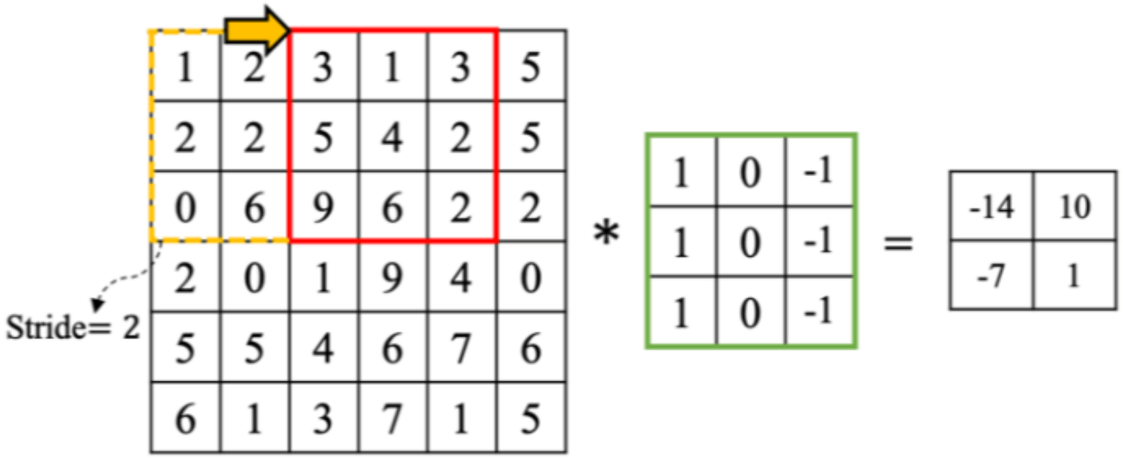
Plug values:

- $= \frac{6-3}{2} + 1 = \frac{3}{2} + 1$
- $= 1.5 + 1 = 2.5$
- But output size must be an integer.
  We always floor the value: $Output\ size = 2$
- **So feature map size = 2 × 2**

(a) Stride = 1



(b) Stride = 2



3. Padding

- **Padding** means adding extra border around the image, usually with zeros.
- Because without padding:
  - output gets smaller very quickly
- With padding:
  - we can keep more border information
  - sometimes keep output size same

Example

  - [1 2]
  - [3 4]
- After zero padding:
  - [0 0 0 0]
  - [0 **1 2** 0]
  - [0 **3 4** 0]
  - [0 0 0 0]

- **Prevent Size Reduction:** Without padding, each convolution shrinks the output size (e.g., **6x6** input with filter **3x3** becomes **4x4**), limiting the depth of the network.
- **Preserve Edge Information:** In "valid" convolution (no padding), corner pixels are processed fewer times than center pixels.
- **Maintain Spatial Dimensions ("Same" Padding):** Padding allows the output image to have the same width and height as the input, which is crucial for building deep networks.

3. Padding

Input Image

Applying padding
of 1 on 3x3

0	0	0	0	0
0				0
0				0
0				0
0	0	0	0	0

Padded Image

3. Padding (How Feature Map)

- **Image = 6 × 6**
- **Filter = 3 × 3**
- **Stride = 1**
- **Padding = 1**
- We add a border of 1 pixel of zeros around the image.
- Original image (6×6) becomes:
- $(n + 2p) \times (n + 2p)$ image after padding
- New size = (6 + 2) × (6 + 2) = 8 × 8
- Padding adds:
  - 1 row on top
  - 1 row at bottom
  - 1 column on left
  - 1 column on right

Compute output size with padding

- Formula:
- $Output = \frac{(n-f+2p)}{s} + 1$
- Plug in values:
  - $n = 6$
  - $f = 3$
  - $p = 1$
  - $s = 1$
- $= \frac{(6-3+2\cdot 1)}{1} + 1$
- $= \frac{(6-3+2)}{1} + 1$
- $= \frac{5}{1} + 1 = 6$
- **Output feature map = 6 × 6**

4. Convolution Feature Map

- The **output** of a convolution layer is called a **feature map**.
- Why “feature” map?
- Because it shows **where a pattern was found** in the image.
- For example, if a filter detects vertical edges, the feature map will highlight places where vertical edges exist.

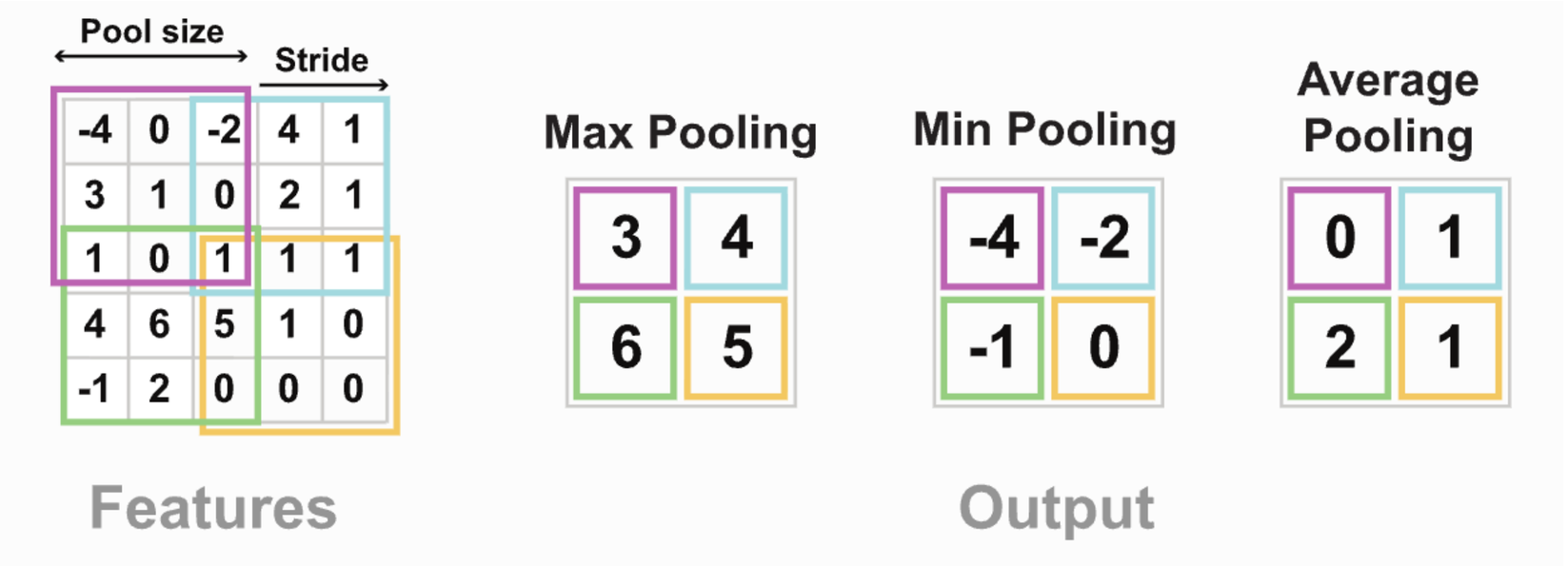
  Feature map = **result after filter scans the image**

Convolution is the operation where the filter slides over the image and creates output values.

- So:
- **filter** = tool
- **convolution** = action of using that tool

5. Pooling

- Pooling **reduces the size** of the feature map to make it smaller, simpler, and faster.
- Think of pooling like **compressing** the image while keeping the **important information**.
  - Pooling does **NOT** have weights.
  - Pooling does **NOT** learn anything.
  - Pooling just **selects values**.
- Two Most Common Types
  - Max Pooling (most used)
  - Average Pooling



5. Pooling

Feature Map

6	6	6	6
4	5	5	4
2	4	4	2
2	4	4	2

Max Pooling

Average Pooling

Sum Pooling

5. Pooling (MAX POOLING)

- Max pooling takes the **largest value** from a small region.
- **Example: 2×2 max pooling**
- Feature map (4×4):
  - 1 3 2 1
  - 5 6 1 2
  - 4 2 8 3
  - 1 1 0 6
- Apply **2×2 pooling with stride = 2**:
- Take each 2×2 block:
- **Block 1:**
  - 1 3
  - 5 6
- max = **6**
- **Block 2:**
  - 2 1
  - 1 2
- max = **2**
- **Block 3:**
  - 4 2
  - 1 1
- max = **4**
- **Block 4:**
  - 8 3
  - 0 6
- max = **8**
- **Final pooled output (2×2):**
  - 6 2
  - 4 8
- It **reduces 4×4 → 2×2** but keeps the strongest features.

5. Pooling (AVERAGE POOLING)

- Instead of the max, you take the **average**.
- Example block:
  - 1 3
  - 5 6
- Average: $\frac{1+3+5+6}{4} = 3.75$

WHY DO WE USE POOLING?

- Pooling helps:
  - Reduce feature map size
  - Reduce number of parameters
  - Reduce computation
  - Reduce overfitting
  - Keep important information
  - When the object moves a little, pooled features remain similar.
- **POOLING PARAMETERS**
- Pooling has:
- **Filter size (window size)** e.g. 2×2, 3×3
- **Stride** (how far we move)
- **No weights**
- **No bias**
- **No learning**

6. Activation Function

- After convolution, we usually apply an activation like **ReLU**:

  ReLU(x) = max(0, x)

- So:
- negative values become 0
- positive values stay
- Example:
- [-2, 3, -1, 5] → [0, 3, 0, 5]
- Why?

- It helps the network learn complex patterns.

7. Flatten

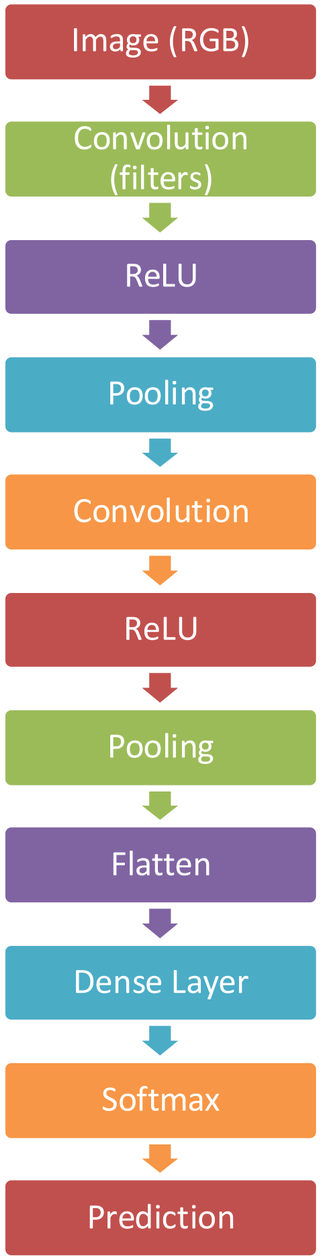
- After many convolution and pooling layers, we convert the output into one long vector.
- Example:
- (5, 5, 64) →1600 values
- This is called **flattening**.
- Then it can go into dense layers.

8. Dense (FC)

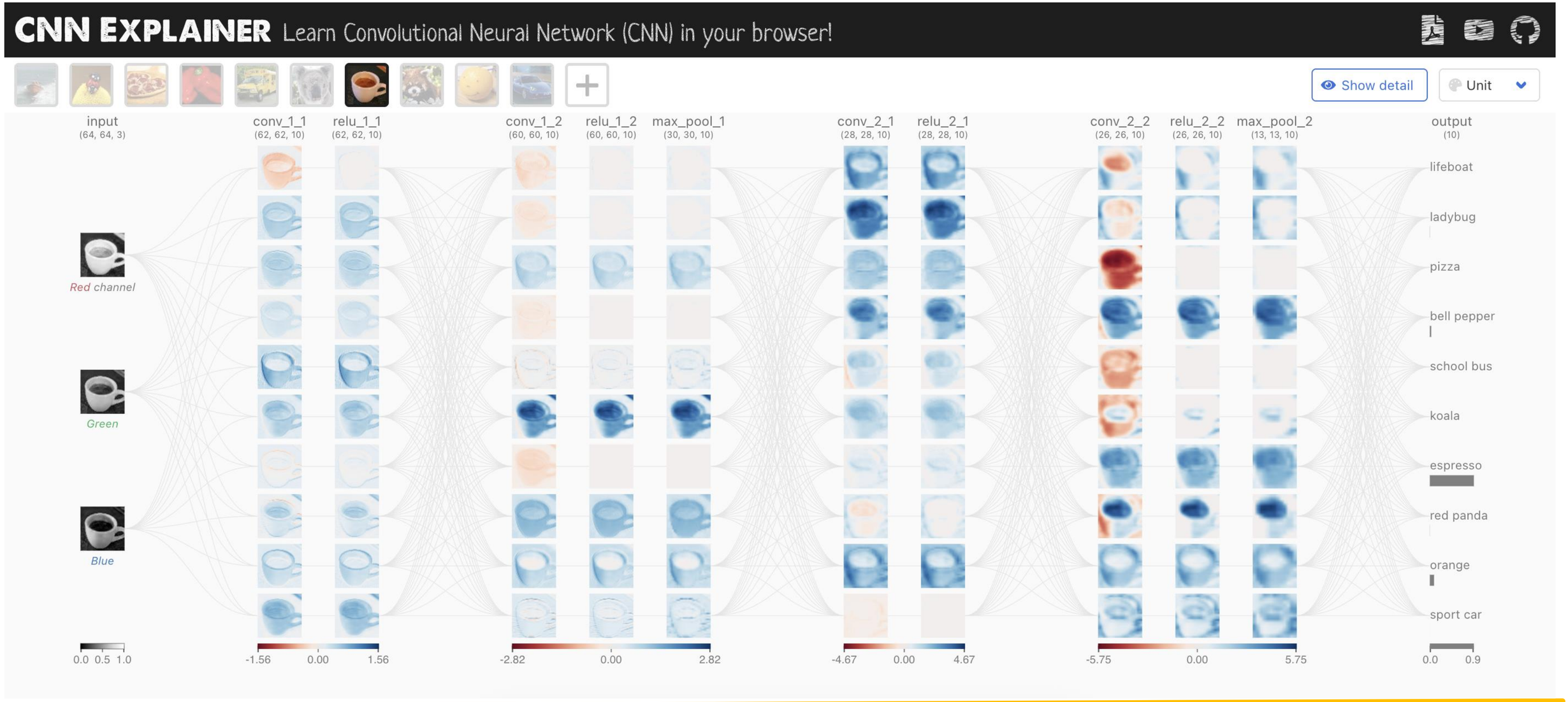
- A **dense layer** is the normal fully connected neural network layer.
- It uses the extracted features to make the final decision.
- Example:
  - cat
  - dog
  - car

Full Flow

- **Input image:** 32 x 32 x 3
- Step-by-step
- **Filter**
  - small scanner looks for pattern
- **Convolution**
  - filter moves over image
- **Feature map**
  - result of scanning
- **ReLU**
  - keeps useful positive values
- **Pooling**
  - makes feature map smaller
- **Repeat**
  - deeper layers learn more complex features
- **Flatten**
  - convert to 1D vector
- **Dense**
  - final classification



CNN Explainer



Most Popular Image Datasets

- **1. MNIST (Handwritten Digits)**
  - The “Hello World” of image classification.
  - 70,000 grayscale digit images (28×28)
  - 10 classes (0–9)
  - https://yann.lecun.com/exdb/mnist/
- **2. CIFAR-10 / CIFAR-100**
  - Very popular for CNN beginners.
  - 32×32 color images
  - CIFAR-10: 10 classes
  - CIFAR-100: 100 classes
  - https://www.cs.toronto.edu/~kriz/cifar.html
  - https://www.kaggle.com/c/cifar-10
- **3. ImageNet**
  - The most influential large-scale image dataset.
  - 14M+ images
    20K+ categories
  - Used in the ImageNet Challenge (ILSVRC)
  - https://www.image-net.org/
- **4. COCO (Common Objects in Context)**
  - Used for detection, segmentation, keypoints.
  - 330,000+ images
  - 80 object categories
  - Captions included
  - https://cocodataset.org/
- **5. Fashion-MNIST**
  - A stronger replacement of MNIST.
  - 70,000 clothing images
  - 10 classes
  - Same format as MNIST
    https://github.com/zalandoresearch/fashion-mnist
- **6. SVHN (Street View House Numbers)**
- Real-world digits from Google Street View.
  - 600,000 labeled digits
  - Harder version of MNIST
  - Color images
  - http://ufldl.stanford.edu/housenumbers/
- **7. PASCAL VOC**
  - Classic dataset for detection & segmentation.
  - 20 object classes
  - Bounding boxes, segmentation masks
  - http://host.robots.ox.ac.uk/pascal/VOC/
- **8. CelebA (Celebrity Faces Attributes)**
  - Used for face detection & attributes.
  - 200,000+ celebrity faces
  - 40 attributes
  - Widely used in GANs (e.g., StyleGAN)
  - http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html
- **9. Tiny ImageNet**
  - Smaller version of ImageNet.
  - 200 classes
  - 100,000 images (64×64)
  - https://www.kaggle.com/c/tiny-imagenet
- **10. Caltech 101 / Caltech 256**
  - Great for object recognition.
  - 101 classes (old version)
  - 256 classes (new)
  - http://www.vision.caltech.edu/Image_Datasets/Caltech101/
- **11. Oxford 102 Flowers**
  - Beautiful dataset for fine-grained classification.
  - 102 flower species
  - ~8,000 images
    https://www.robots.ox.ac.uk/~vgg/data/flowers/102/
- **12. FER-2013 (Facial Expression Recognition)**
  - Emotion classification via images.
  - 7 emotion classes
  - 48×48 grayscale faces
  - https://www.kaggle.com/datasets/msambare/fer2013
- **13. COIL-100**
  - Object images from multiple angles.
  - 7,200 images
  - 100 daily objects
  - http://www1.cs.columbia.edu/CAVE/software/softlib/coil-100.php
- **14. MIT Indoor Scenes**
  - Scene recognition dataset.
  - 67 indoor scene categories
  - 15,000 images
  - http://web.mit.edu/torralba/www/indoor.html

TOP RECOMMENDATION

- **(If You’re Learning CNNs)**
- **Start with:**
  - MNIST
  - CIFAR-10
  - Fashion-MNIST
- **Then move to:**
  - ImageNet
  - COCO
  - CelebA

TOP RECOMMENDATION

- **(If You’re Learning CNNs)**
- **Start with:**
  - MNIST
  - CIFAR-10
  - Fashion-MNIST
- **Then move to:**
  - ImageNet
  - COCO
  - CelebA

Most Popular CNN Architectures

- **1. LeNet-5 (1998)**
  - First successful CNN
  - Used for handwritten digit recognition
  - Simple: Conv → Pool → Conv → FC
- **2. AlexNet (2012)**
  - Revolutionized deep learning
  - Won ImageNet 2012
  - Introduced **ReLU**, **Dropout**, GPU training
- **3. ZFNet (2013)**
  - Improved AlexNet
  - Tuned stride & filter sizes
  - Introduced feature map visualization
- **4. VGG-16 / VGG-19 (2014)**
  - Very deep: 16–19 layers
  - All **3×3 filters**
  - Clean and simple architecture
  - **5. GoogLeNet / Inception (2014)**
  - Introduced **Inception modules** (1×1, 3×3, 5×5 filters in parallel)
  - Very efficient (only ~6M params)
- **6. ResNet (2015)**
  - Introduced **skip connections**
  - Solved vanishing gradients
  - Enabled very deep networks (50, 101, 152 layers)
- **7. DenseNet (2017)**
  - Each layer connects to **every later layer**
  - Very efficient + great gradient flow
  - Used for high accuracy with fewer parameters
- **8. MobileNet (2017)**
  - Designed for mobile/edge devices
  - Uses **depthwise separable convolutions**
  - Very fast & lightweight
- **9. Inception-v3/v4**
  - Improved GoogLeNet
  - Factorized convolutions for efficiency
  - Better accuracy with fewer FLOPs (Floating Point Operations)
- **10. EfficientNet (2019)**
  - Scales depth, width, and resolution **together**
  - Best accuracy-to-efficiency ratio
  - State-of-the-art performance for years

Task Instructions (Take Quiz Next Class)

- **Choose a dataset**
  - MNIST or CIFAR-10
  - Load train/test splits.
- **Preprocess images**
  - Normalize pixel values (0–1)
  - Reshape if required.
- **Build a CNN (minimum layers)**
  - Conv Layer (3×3)
  - ReLU
  - MaxPool (2×2)
  - Conv Layer (3×3)
  - ReLU
  - MaxPool (2×2)
  - Flatten
  - Dense Layer
  - Softmax Output
- **Train the model**
  - Use cross-entropy loss
  - Train for 5–10 epochs
  - Monitor accuracy
- **Evaluate**
  - Report test accuracy
  - Show confusion matrix
  - Display 5 correct & 5 wrong predictions
- **Submission**
  - Code notebook
  - Model architecture summary
  - Results (accuracy + sample outputs)